Pre-commit Hook Setup for Protected Branches

This document explains how to set up a pre-commit hook to prevent accidental commits to protected branches.

What are Protected Branches?

Protected branches are critical branches that should not receive direct commits. The branch management scripts (`mkbranch` and `rmbranch`) prevent deletion of:

- `main`

Setting Up a Pre-commit Hook

A pre-commit hook prevents commits to these protected branches before they're pushed to remote.

1. Create the hook file

Create `.git/hooks/pre-commit` in your repository:

```
touch .git/hooks/pre-commit
chmod +x .git/hooks/pre-commit
```

2. Add the hook script

Edit `.git/hooks/pre-commit` and add:

```
#!/usr/bin/env bash

# Pre-commit hook to prevent commits to protected branches
# This hook runs before every commit to check the current branch

PROTECTED_BRANCHES=("main" "master" "develop" "development")

# Get current branch
CURRENT_BRANCH=$(git rev-parse --abbrev-ref HEAD)

# Check if current branch is protected
for protected in "${PROTECTED_BRANCHES[@]}"; do
  if [[ "$CURRENT_BRANCH" == "$protected" ]]; then
    echo "Error: You are trying to commit to the protected branch '$CURRENT_BRANCH'"
   echo "Please create a feature branch using: scripts/bin/mkbranch -r <branchname> $CUR
    exit 1
  fi
done

exit 0
```

3. Make it executable

```
chmod +x .git/hooks/pre-commit
```

Alternative: Shared Hook Setup

For teams, you can store hooks in version control and set them up automatically:

1. Create a hooks directory in your repository

```
mkdir -p .githooks
```

2. Create the pre-commit hook

Save the script above to `.githooks/pre-commit` and make it executable:

```
chmod +x .githooks/pre-commit
```

3. Configure Git to use this directory

```
git config core.hooksPath .githooks
```

Or configure it globally for your user:

```
git config --global core.hooksPath ~/.githooks
```

4. Commit the hooks directory

```
git add .githooks/
git commit -m "Add pre-commit hooks for protected branches"
```

How It Works

When you try to commit on a protected branch:

```
$ git commit -m "some change"
Error: You are trying to commit to the protected branch 'main'
Please create a feature branch using: scripts/bin/mkbranch -r <branchname> main
```

The commit is rejected, and you must:

1. Switch to or create a feature branch
2. Make your changes on that branch
3. Create a pull request for review

Recommended Workflow

1. **Create a feature branch:**

   ```
   scripts/bin/mkbranch -r my-feature main
   ```

2. **Make your changes:**

   ```
   git add .
   git commit -m "Add my feature"
   ```

3. **Push to remote:**

   ```
   git push origin patch/my-feature
   ```

4. **Create a pull request** on GitHub/GitLab

5. **After approval, merge and clean up:**

   ```
   scripts/bin/rmbranch -a patch/my-feature
   ```

Troubleshooting

Hook is not running

Check that: - The hook file is executable: `ls -la .git/hooks/pre-commit` - The shebang line is correct: `#!/usr/bin/env bash` - Git hooks are enabled in your repository

Need to bypass the hook (not recommended)

Use the `--no-verify` flag to skip hooks:

```
git commit --no-verify -m "message"
```

Note: This should only be used in emergencies and is not recommended for shared repositories.

Additional Server-Side Protection

For maximum protection, configure branch protection rules in your repository settings:

1. Go to Repository Settings → Branches
2. Add a branch protection rule for `main`, `master`, `develop`, etc.
3. Enable:
   - Require pull request reviews
   - Require status checks to pass
   - Dismiss stale pull request approvals
   - Require branches to be up to date before merging

This prevents any direct pushes to protected branches, even if someone bypasses the local hook.